

Projet L3 : PacMan en réseau

Thomas Coutant | Benoît Litampha | Auriane Peter

Université Marie & Louis Pasteur, UFR Sciences et Techniques

Licence CMI Informatique – 3^{ème} année
2025–2026

Encadrant : Julien Bernard



Plan

- 1 Présentation & organisation
- 2 Server
- 3 Client
- 4 Réseau
- 5 Démonstration

- Contexte du projet
- Objectifs
- Organisation

Plan

1 Présentation & organisation

2 Server

3 Client

4 Réseau

5 Démonstration

- **Git** pour le versionnement du projet
- **Discord** pour la communication entre les membres du groupe et avec le tuteur
- **réunions régulières** (toutes les 1 à 2 semaines) pour faire le point sur l'avancement
- Communication **quotidienne** entre les membres pour coordonner le travail



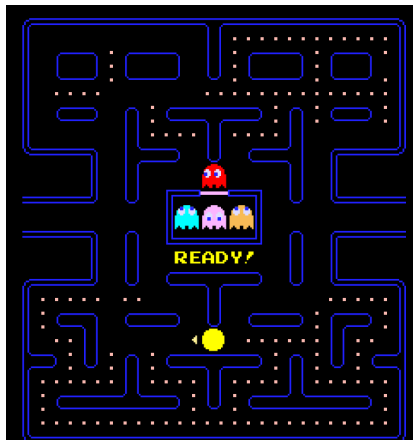
Présentation du jeu Pac-Man

Jeu d'arcade apparu dans les **années 1980**

- Pac-Man (le joueur) évolue dans un **labyrinthe**
- Objectif : manger toutes les **pac-gommes**
- Éviter les **fantômes**
- Certaines pac-gommes permettent de **manger les fantômes**

Adaptation du projet :

- 1 joueur Pac-Man
- plusieurs joueurs Fantômes
- fantômes contrôlés par **IA** si joueurs insuffisants



Jeu original Pac-Man

■ **Thomas Coutant**

- Développement du serveur
- Conception de la logique du jeu
- Prototype initial client–serveur

■ **Auriane Peter**

- Développement du client
- Interface et interactions joueur

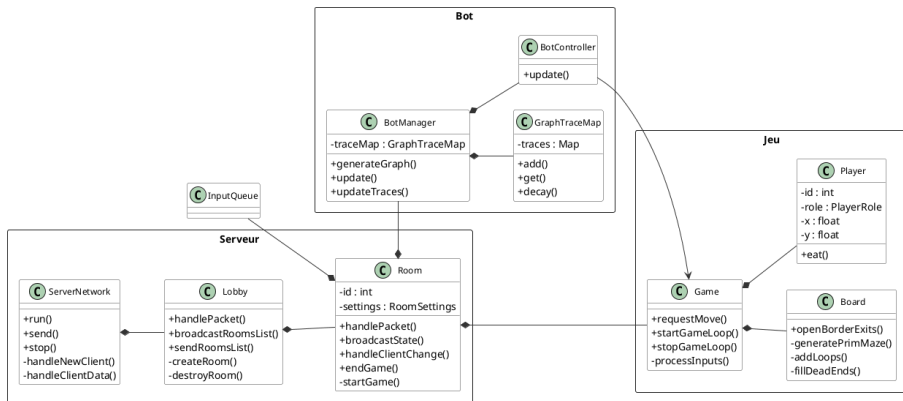
■ **Benoît Litampha**

- Communication client–serveur
- Support sur client et serveur

Plan

- 1 Présentation & organisation
- 2 Server
- 3 Client
- 4 Réseau
- 5 Démonstration

Architecture du serveur



Client → Server → Lobby → Room → Game

Boucle de jeu

Serveur autoritaire

- Le **serveur décide de tout**
- Les **clients affichent uniquement l'état reçu**
- Permet d'**éviter la triche** et les désynchronisations

Tick serveur

- La boucle de jeu fonctionne avec des **misés à jour périodiques**
- Un **thread serveur** met à jour l'état du jeu :
 - positions des joueurs et fantômes
 - pac-gommes
 - timers
 - score et conditions de victoire

États de la partie

- Avant la partie (préparation)
- Partie en cours
- Fin de partie

IA des fantômes - Principe

Avoir des fantômes capables de s'adapter au nombre de joueurs et de produire une forme d'**intelligence collective**.

Inspiration : les fourmis

- Dans la nature, les fourmis utilisent des **phéromones** pour communiquer.
- Chaque fourmi laisse une trace que les autres peuvent suivre.

Adaptation dans le jeu

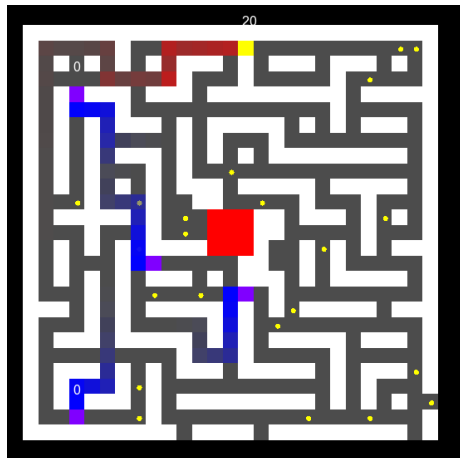
- Les entités laissent des **traces** sur le plateau.
- Chaque trace possède :
 - une **intensité** qui diminue avec le temps
 - un **propriétaire**
 - une **couleur**
- **Rouge** : Pac-Man | **Bleu** : fantômes

IA des fantômes - Prise de décision

Les fantômes choisissent leur déplacement selon le score :

$$\text{Score} = \text{Attraction} - \text{Répulsions}$$

- Les traces de **Pac-Man** créent une **attraction**
- Les traces des **fantômes** créent une **répulsion**
- La **répulsion** est plus forte pour les **traces du fantôme lui-même**



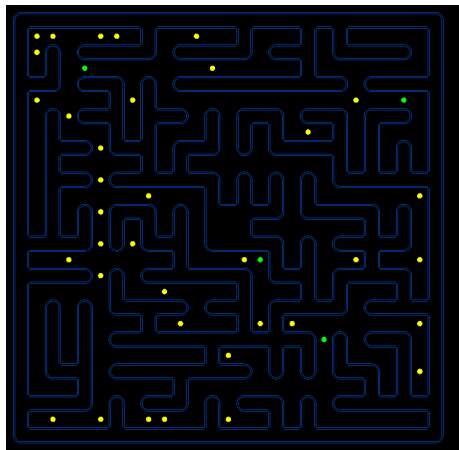
Génération procédurale du plateau

Variante de l'algorithme de Prim

- Plateau initialement rempli de murs
- Choix d'une case de départ
- Ajout des murs adjacents dans une liste
- Ouverture progressive de murs aléatoires

Résultat :

- Labyrinthe connexe
- Aucune zone isolée

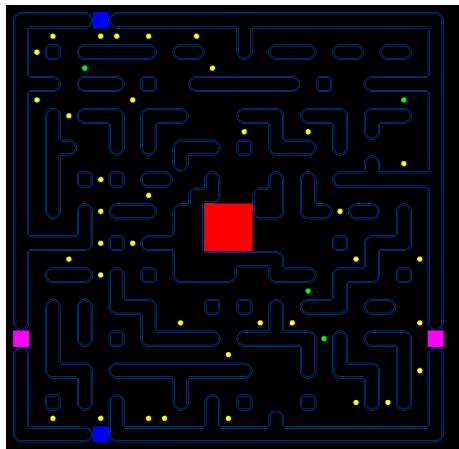


Contraintes du jeu

- Zone centrale fixe : **hut** 3×3
- Vérification de l'accès aux **coins de départ**

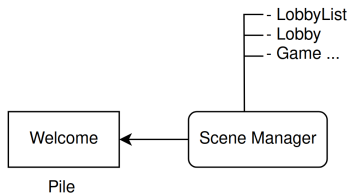
Améliorations

- Ajout de **cycles**
- Réduction des **culs-de-sac**
- Ajout de **portails latéraux**



Plan

- 1 Présentation & organisation
- 2 Server
- 3 Client**
- 4 Réseau
- 5 Démonstration



Fonctionnement du SceneManager

Le SceneManager gère quelle scène s'affiche :

- Chaque étape de jeu : **une scène**
- La scène se **gère elle même**
- **Séparation** entre logique / rendu

Met aussi en place le réseau

Les scènes envoient les messages au serveur

Deux modèles :

- événementiel (menus, lobby)
- périodique (mouvements en jeu)

Principe d'envoi est **le même**

L'envoi transmet l'intention du client au serveur

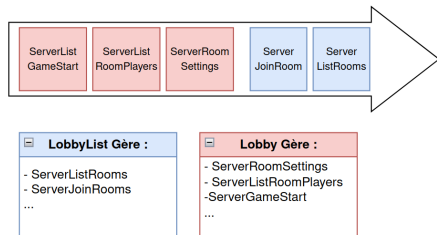
Reception

Réception dans un **Thread dédié**

Paquets stockés dans une file
partagée :

- Scène consomme paquets **selon leur type**
- Gère seulement ce **qui les concerne**

La scène se met à jour en fonction de
ce qu'elle a reçu



Système d'**Actions** :

Mapping clavier -> actions (up, down, left, right)

Événements gérés par chaque scène :

- Passage de la souris
- Clics de la souris
- Redimensionnement de la fenêtre

Au final, server determine ce qui est effectif

Les entités gèrent le **rendu**

Les scènes gèrent la **logique**

Chaque scène a une entité

L'entité traduit la logique décidée en éléments graphiques concrets

Elle interagit directement avec le client : lien clics clients - logique événement

Plan

- 1 Présentation & organisation
- 2 Server
- 3 Client
- 4 Réseau**
- 5 Démonstration

Communication avec socket

- Choix entre UDP et TCP : TCP choisit pour sa fiabilité
- Ip et port précisé en ligne de commande

Classes Spécialisées

- ServerNetwork pour la gestion des nouveaux client et des packets
- ClientNetwork pour la gestions des packets

Multi-Threading

- Fonctions de ServerNetwork et ClientNetwork exécuté dans un thread
- Dépilement des packets dans la boucle principale

Structures communes

- Utilisé à la fois par le client et le serveur
- Peut être généré dynamiquement par un objet du serveur

Structures de Communication

- Utilisé pour les échanges réseaux
- Un champ *gf* : *id* pour les identifier lors de la désérialisation
- Nomenclature spécifique :
 - ServerXXX = envoyé par le server
 - ClientXXX = envoyé par le client

```
struct PlayerData
{
    uint32_t id;
    float x, y;
    uint32_t color;
    std::string name;
    PlayerRole role;
    int score;
    bool ready = false;
    unsigned int hp = 0;
};
```

```
struct ServerListRoomPlayers
{
    static constexpr gf::Id type =
        "ServerListRoomPlayers"
        _id;
    std::vector<PlayerData>
        players;
};
```

Opérateur

- Operateur | avec un type Archive

Sérialisation/Envoi

- Fonction *gf : :Packet : :is(obj)* pour sérialiser en suite d'octet
- Fonction
gf : :TcpSocket : :sendPacket(packet)
pour envoyer le packet

Désérialisation

- Switch case pour identifier le type
- Fonction *gf : :Packet : :as(objType)*
pour désérialiser

```
template <typename Archive>
Archive &operator|(Archive &ar,
    PlayerData &data)
{
    return ar | data.id
        | data.x
        | data.y
        | data.color
        | data.name
        | data.role
        | data.score
        | data.ready
        | data.hp;
}
```


Exemple concret : Envoi de la liste des joueurs

Dans le serveur

```
void Room::broadcastRoomPlayers() {  
    ServerListRoomPlayers list;  
    for (uint32_t pid : players) {  
        PlayerData pdata;  
        ...  
        list.players.push_back(pdata);  
    }  
    gf::Packet packet;  
    packet.is(list);  
    for (uint32_t pid : players) {  
        network.send(pid, packet);  
    }  
}
```

Dans le client

```
void LobbyScene::doUpdate(gf::Time)  
{  
    gf::Packet packet;  
    while (m_game.tryPopPacket(packet)) {  
        switch (packet.getType()) {  
            case ServerRoomSettings::type:  
                ...  
            case ServerListRoomPlayers::type:  
                {  
                    auto data =  
                        packet.as<ServerListRoomPlayers>();  
                    setPlayers(data.players);  
                    break;  
                }  
            ...  
        }  
    }  
}
```

Conclusion

TODO

TODO

TODO